



THM

TECHNISCHE HOCHSCHULE MITTELHESSEN

**CAMPUS
FRIEDBERG**

IEM

Informationstechnik-
Elektrotechnik-Mechatronik

Technische Hochschule Mittelhessen

Observation of IPv6 Segment Routing (SRv6) Network Routing in Simulated Lab Environments

Student's Name: Shibichakaravarthy Senthil

Supervisor: Prof. Dr. rer. nat. Dieter Baums

Co-Supervisor: Prof. Dr. rer. nat. Christian Prause

Contents

1	Introduction	5
1.1	Thesis Description	6
1.2	Research Framework	6
1.2.1	Hypothesis	6
1.2.2	Aim	6
1.2.3	Goals	7
1.3	Thesis Planning	7
2	Literature Review	9
3	Test conducted in Cisco Packet Tracer	12
4	Topology	14
4.1	Overview	14
4.2	Impairments and SR capability	15
4.3	Emulator overview	16
5	Tools and Pre-Tests	17
5.1	Tools	17
5.2	pre-tests	17
5.3	Policy based programming	20
6	Ideas and Next Step	22
	Bibliography	23

List of Figures

Figure 4.1: Clos architecture 16

Figure 5.1: SRv6 over ISIS infos from Ingress CLI 18

Figure 5.2: SR capability of SR nodes 18

Figure 5.3: packets from outside 19

Figure 5.4: packets from Inside 19

Figure 5.5: 4 SIDs packet 20

List of Tables

1 Introduction

The rapid growth of devices connected to the internet has placed increasing pressure on modern network infrastructures. As a result, there is a stronger need for routing solutions that are both scalable and flexible, especially to support the wide range of emerging applications. Traditional IPv4 routing struggles in this regard due to its limited 32-bit address space. IPv6 addresses these limitations by introducing a much larger 128-bit address space, along with a simplified header format and support for extension headers, which together allow for more efficient and scalable packet processing.

SRv6 (Segment Routing over IPv6) is a new paradigm of packet forwarding that leverages the capabilities of IPv6. Unlike traditional hop-by-hop routing, SRv6 allows the encoding of forwarding instructions directly into the packet using the Segment Routing Header (SRH). This enables a source node to define a specific path as an ordered list of Segment Identifiers (SIDs), creating a programmable networking environment where routing and service operations are expressed as a sequence of instructions. Furthermore, SRv6 is backward compatible with existing IPv6 infrastructures. Since SIDs are encoded as standard IPv6 addresses, intermediate nodes that do not support SRv6 can still forward packets using conventional IPv6 mechanisms. This feature enables incremental deployment of SRv6 with minimal changes to current network architectures.

Recent developments, particularly the introduction of micro-Segment Identifiers (uSID), aim to improve SRv6 scalability by compressing segment lists and reducing header overhead. Although SRv6 has gained significant attention in both academia and industry, experimental studies in controlled and reproducible environments remain limited. In particular, forwarding behaviour, performance under varying traffic conditions, and the practical implications of uSID-based encoding are not yet fully understood in emulated or containerised network environments.

This thesis addresses this gap by designing and implementing a containerised SRv6 experimentation environment using FRRouting and ContainerLab. The objective of this work is to analyse SRv6 routing behaviour in detail, evaluate the efficiency of uSID-based compression, and compare practical implementation results with theoretical specifications defined in relevant standards such as RFC 8754 and RFC 9800.

1.1 Thesis Description

The master's thesis mainly focuses on analyzing the "Network programming" capabilities of Segment Routing over IPv6 (SRv6) and its performance and architectural design. In traditional IP forwarding, routers follow the control-plane instructions such as ISIS and BGP to transfer the packets to the destination address. This helps routers to choose shortest paths and forward is performed hop-by-hop using destination-based longest prefix matching, with routing agility distributed across all nodes. But SRv6 offers source-driven forwarding algorithms in which an ordered set of instructions, described as Segment Identifiers (SIDs), is encoded directly within the IPv6 packet extension header. It uses packet carry instructions, so routers no longer compute per-flow path decisions but execute instructions encoded in the packet. Routers just execute the instructions without computing the per-flow path decisions. The micro-SID mechanism introduces a compressed encoding architecture that enables multiple SIDs to be embedded within a single IPv6 packet. The network system moves to a distributed instruction execution system from a simple forwarding fabric system. This method is considered a form of "Network Programming". The primary objective of this thesis is to observe the SRv6 mechanisms in a simulated model and analyze their behavior forwarding efficiency, scalability, and path determinism. This knowledge can be used in the THM laboratory for future teaching or research purposes or laboratory development.

1.2 Research Framework

1.2.1 Hypothesis

The network programming paradigm known as Segment Routing using IPv6 (SRv6), particularly with micro-segment identifiers (uSID), is expected to provide improved efficiency, scalability, and determinism in packet forwarding. This is due to its ability to embed execution instructions directly within packet headers. As a result, SRv6 reduces the complexity associated with traditional hop-by-hop IP forwarding mechanisms.

1.2.2 Aim

This thesis aims to design and analyse an SRv6-based network that incorporates micro-segment identifiers (uSID) within an emulated laboratory environment. The study focuses on evaluating forwarding behaviour, packet processing mechanisms, segment routing operations, and header efficiency. The resulting framework and findings are intended to support further research, experimentation, and educational applications in SRv6-based network

environments.

1.2.3 Goals

The specific goals of this thesis are as follows:

1. Design and deploy an SRv6-based network topology in an emulated environment using container-based tools.
2. Implement and validate SRv6 routing using both standard Segment Identifier (SID) and micro-SID (uSID) approaches.
3. Analyse uSID-based forwarding behaviour during packet processing, including SID execution and path progression.
4. Examine segment list compression mechanisms and quantify their impact on packet header structure and size.
5. Investigate the support of uSID for programmable network functions, such as traffic steering and service chaining.
6. Perform a comparative evaluation of standard SRv6 and uSID in terms of header overhead, scalability, and operational behaviour.

1.3 Thesis Planning

The experimental environment developed in this thesis emulates a data centre networking architecture using Segment Routing over IPv6 (SRv6) within a controlled and reproducible laboratory setup. The topology consists of approximately twelve nodes representing essential components of a modern data centre network. These include a client node, an ingress router acting as the SRv6 headend, multiple core routers forming the SR domain, network functions such as a firewall and an intrusion detection system (IDS), an egress router, a load balancer, application servers, and a telemetry server for monitoring and data collection.

Traffic generated by the client is encapsulated at the ingress router using a Segment Routing Header (SRH), enabling explicit path control and service chaining across the network. This allows traffic to traverse security functions and multiple routing paths in order to evaluate traffic engineering capabilities and path determinism.

The environment is implemented using Containerlab for topology deployment, FRRouting (FRR) with Linux SRv6 support for routing functionality, Wireshark for packet-level analysis, and Prometheus with Grafana for performance monitoring and visualisation. Various

experimental scenarios are conducted, including baseline performance evaluation, traffic engineering, service chaining, load balancing, and analysis of segment list depth and header overhead, with particular emphasis on uSID-based compression.

Each experiment is performed under controlled conditions. The collected metrics—such as latency, throughput, packet loss, and resource utilisation—are analysed using statistical methods to evaluate SRv6 forwarding behaviour, scalability, and efficiency, and to compare the results with theoretical specifications.

2 Literature Review

In modern networks, many protocols work together, and sometimes this makes internet routing more complex and increases operational complexity. IPv4 is incompatible with these requirements due to address exhaustion, limited extensibility, and lack of design for modern programmability. Nowadays devices that connect to the internet almost exceed available IPv4 addresses. Core devices need to process the variable length of the IPv4 header and recompute the checksum for error detection. It adds processing steps. So IPv6 addresses were introduced to resolve these limitations. Unlike an IPv4 address, an IPv6 address has a 128-bit size, and its address space is enormous compared to Internet-connected devices. IPv6 has only a simple and basic header (a fixed 40-byte header)[1]. IPv6 is more efficient because it doesn't do header checksum; it reduces the processing overhead. It moves complex optional headers to extension headers[1]. It has an extension header to add more information, and this facilitates solutions for modern routing problems, such as improving packet routing efficiency and enabling better support for new protocols. IPv6 uses the Next Header field to chain extension headers, with each header identifying the type of header that comes next[1]. This flexible structure allows the integration of advanced mechanisms such as Segment Routing (SR).

Segment routing was introduced to lighten the protocols' complexity, and it can be implemented by these IPv6 extension headers[2]. It is a modern networking architecture that minimises the complexity of traffic engineering by encoding the forwarding path directly into the packet[2]. Segment Routing was introduced in IPv6 as extension header. The Segment Routing Header (SRH) is defined as an IPv6 extension header that enables Segment Routing functionality[3]. It carries a sequence of segments, where each segment represents a specific instruction related to network topology, services, or packet processing[2]. These segments are identified using Segment Identifiers (SIDs), which are typically expressed as IPv6 addresses. Each SID defines a step in the forwarding process that the packet must follow[2]. The currently active segment is stored in the destination address field, while the remaining SIDs are maintained within the SRH. As the packet moves through the network, each node processes the active segment and updates the header accordingly. This enables routers to process packets based on explicit instruction rather than maintaining per-flow states[2]. SRv6 supports different types of segments that define how packets must be handled. Prefix SIDs describe the network prefix and are configured globally forward by Shortest Path First (SPF). But Node-SIDs represent the specific router in the network and are globally configured. Any-cast-SIDs help packets reach the nearest router with the same destination address. Adjacency-SIDs force packets to forward to a specific interface to reach a specific node, and only a specific local node can understand the SID and work

on the process. This feature provides the ability to control the path decision by overriding the default shortest path routing. An SR policy defines a path as a list of segments. These policies are manually or dynamically created by the operators or SDN controllers. A Binding SID represents an entire Segment Routing policy as a single identifier. It improves scalability and secures the path from the external users. Users will remain unaffected even if the operator alters the policy, as they can only view binding SIDs. SRv6 operates in only trusted domains and requires strict filters at the edges of the domain to prevent the attackers from injecting the packets to distribute the forwarding. It is not a new protocol, and it is a new mechanism to make routing simpler and better[2]. RFC 8754 shares the details of the data plane and how SRv6 is processed by the nodes.

RFC 8754 defines the Segment Routing Header (SRH) as an IPv6 extension header that contains a list of segments describing the path a packet is expected to take[3]. This paradigm supports stateless forwarding in the network, so it simplifies the data plane and enhances scalability[2]. The SRH has several important fields: Next Header, Hdr Ext Len, Routing Type, Segments Left, Last Entry, Flags, Tag, and the Segment List[3]. The segment list is arranged in reverse order, so its first segment represents the final instruction of the policy. The Segment Left field acts as a pointer indicating the next segment to be processed, and it is decremented at each segment endpoint node. This RFC also introduces optional Type-Length-Value (TLV) objects within the SRH, which provide additional information for segment processing. Among these, padding TLVs provide proper alignment of the header, and the HMAC TLV ensures authentication and integrity verification of the SRH. This HMAC mechanism verifies that the segment list is not modified from outside of the domain during transit and that packets come from authorised nodes. By design, the SR domain provides additional security for the routing; only trusted nodes can use Segment Identifiers (SIDs). This security provides filtering of malicious packets at the headend node (ingress) and access control lists (ACLs) to prevent unauthorised use of SIDs. In addition, MTU handling, ICMP processing, and load balancing using the IPv6 flow label are crucial for better operation. In the end, SRv6 provides a scalable, flexible, and mainly programmable networking architecture by utilising IPv6 capabilities and simplifying traffic engineering[3]. RFC 8986 defines the SRv6 programmable capabilities in detail.

RFC 8986 describes the SRv6 network programming model, where Segment Identifiers (SIDs) represent specific instructions that are executed at each node, enabling a programmable networking paradigm. The Segment Routing Header (SRH) carries a list of SIDs, and each node processes the active SID by executing its associated locally configured behavior[4]. This RFC defines various endpoint behaviors such as End, End.X, and End.T, along with their variants including End.DX4, End.DT6, End.DX2, and End.DT2U, which support Layer 3 forwarding, Layer 2 services, decapsulation, and VPN functionalities. In addition, policy-related behaviors such as End.B6.Encaps and End.BM are used to steer

traffic along predefined paths, enabling efficient traffic engineering without maintaining per-flow state in the network core. To improve forwarding efficiency, methods such as Penultimate Segment Pop (PSP), Ultimate Segment Pop (USP), and Ultimate Segment Decapsulation (USD) remove the SRH at different stages of packet processing. Headend behaviors such as H.Encaps and H.Encaps.L2 define how IPv4, IPv6, or Ethernet traffic is encapsulated into SRv6 packets, allowing seamless integration of different traffic types. While the RFC primarily focuses on data-plane behaviors, it also references control-plane mechanisms such as IGP and BGP for SID distribution. Security considerations emphasize operation within a trusted SR domain, proper SID allocation, and filtering policies to prevent unauthorized use of SIDs[4]. Overall, SRv6 network programming transforms the network into a flexible and scalable system by associating instructions with SIDs and enabling efficient service delivery.

3 Test conducted in Cisco Packet Tracer

The experiment was conducted in a simulated environment using Cisco Packet Tracer, where an IPv6-based multipath topology was designed to emulate SRv6-like behaviour. The network consisted of routers R1 to R6, end devices, and a server, all interconnected using a predefined IPv6 addressing scheme. The topology followed a dual-path structure, allowing traffic to traverse multiple routes between the client and the server.

Initially, an intra-domain routing setup was established using OSPFv3 as the control plane and IPv6 CEF for packet forwarding. R1 was configured as the ingress router, R2 to R5 as transit routers, and R6 as the egress router. In the first scenario, OSPFv3 with equal-cost multipath (ECMP) was enabled. Traffic was distributed across multiple available paths, resulting in load balancing. However, the forwarding behaviour was non-deterministic, as the path selection varied depending on the hashing mechanism.

In the second scenario, OSPF cost values were manually adjusted to create a preferred primary path and a secondary backup path. Under normal conditions, traffic consistently followed the lower-cost path. Then one of the link from SPF (Shortest Path First) has been disabled, traffic was successfully redirected to the backup path after convergence. This approach improved determinism compared to ECMP but still depended on OSPF convergence time.

In the third scenario, a static route was configured at the ingress router (R1) to override OSPF decisions and enforce a specific forwarding path. As a result, traffic consistently followed the predefined route regardless of the underlying OSPF topology. This behaviour closely resembled SRv6-style path steering, where forwarding decisions are defined at the ingress. The results showed highly deterministic forwarding with minimal dependence on dynamic routing changes.

Convergence testing was performed by generating continuous ICMP traffic between the client and the server while introducing link failures. In the ECMP scenario, traffic experienced brief disruptions, with recovery dependent on OSPF reconvergence. In the cost-based routing scenario, failover occurred after convergence delays, leading to temporary packet loss. In contrast, the static route scenario showed more stable behaviour, although it lacked automatic adaptability to topology changes.

In a second part of the experiment, an extended topology with routers R1 to R7 was used to implement policy-based traffic steering. At router R3, specific traffic flows were matched using access control logic and redirected through an alternate path. Initially, traffic followed the default shortest path (R2–R3–R4–R5). After applying the policy, the path changed to

R6–R7, demonstrating explicit control over traffic forwarding. This confirmed that selective path steering could be achieved based on predefined rules.

Additional validation was carried out using traceroute and ping. The results verified path changes before and after applying routing policies. MTU testing further showed that packets exceeding the configured size limits resulted in transmission failure, highlighting the importance of proper MTU configuration in multi-hop environments.

Overall, the experiment demonstrated that although native SRv6 features such as SRH and SID processing were not available, similar behavioural characteristics such as ingress-based path control, deterministic forwarding, and traffic steering could be effectively emulated using conventional IPv6 routing mechanisms combined with static and policy-based configurations.

4 Topology

4.1 Overview

There are plenty of core nodes in this network with some edge nodes and other service nodes like firewalls and intrusion detection systems which will be added later with a restricted number of host devices. All nodes are configured with pure IPv6 addresses and routing with the IGP ISIS protocol. The focus of this thesis is on the network layer, which is concerned with Layer 3 protocols. This network is based on the Clos architecture concept which is good for scalability. This is leaf-spine technology and is used mostly in data centers. Edge devices connect to leaf nodes and spine devices connect the leaf routers to maximise the use of bandwidth. Moreover, every leaf node is connected to spine nodes, which induces an uniform-cost path. A network created through analogous design. This network disregards equal-cost paths to accurately assess SRv6 behaviours.

The topology is organized into the following layers:

- **Ingress layer** – edge node for incoming traffic
- **Leaf layer (R1, R4)** – performs initial traffic distribution
- **Spine layer (R2, R3, R7)** – represents the core network
- **Exit Leaf layer (R6, R8)** – acts as a bottleneck before reaching the destination
- **Egress layer** – delivers traffic to the final destination

This layered approach helps model traffic flow across different stages of the network, allowing controlled experimentation on how packets traverse from source to destination.

Simulators produce unrealistic results if impairments are not incorporated. Delay, queue for packet loss, and TCP congestion management are implemented through the tools supplied by the Linux community. These techniques replicate those deficiencies through software.

Traffic control (**tc**) mechanisms are applied within each container using layered queuing disciplines:

- **Token Bucket Filter (TBF)** limits bandwidth to model transmission delay
 - **NETEM** introduces propagation delay and jitter with variable latency
 - It delineates the classical delay components: transmission, propagation, queuing, and
-

processing delays. The bandwidth constraint was modified from static to dynamic to enhance realism. This incorporates time-varying network circumstances and produces traffic variations similar to those in actual networks. Although no mathematical loss models are expressly utilised, the integration of RED and dynamic changes inherently produces erratic packet loss patterns [5] [6] [7].

4.2 Impairments and SR capability

At the end of the client host delay might be low, So in leaf routers that are near to client side are configured with 2 to 3.5 ms. Spine routers from the core has been configured with above medium delay to represent the high usage of bandwidth. At the another end of leaf routers configured with high delay and usually near to the server side bottleneck does happen. This distribution reflects real-world behavior, where delay and variability increase toward the core and near congestion points. Now there are two servers with different destination unicast IPv6 addresses and follow different forwarding policies. The network is fully SR-capable, but no SR policies have been applied yet for one destination that is backup server. This SRv6 use SPF to reach the destination similar to traditional IPv6 forwarding.

SRv6 has been enabled on all routers, and SID information is distributed using IS-IS. The control plane successfully advertises:

- SRv6 locators
- End SIDs (node-level reachability)
- End.X SIDs (adjacency-based forwarding)

The locator prefix `fc00:1000:n::/48` is consistently used across the network.

Each SID is structured based on defined fields, including locator, node identifier, and function. This aligns with SRv6 SID representation, where an IPv6 address encodes both location and behavior.

All advertised routes currently use SPF with a metric of zero, confirming that no traffic engineering policies are active.

Each SID is 128 bits (16 bytes). To improve efficiency, **micro SIDs (uSIDs)** can be used to encode multiple segments within a smaller space.

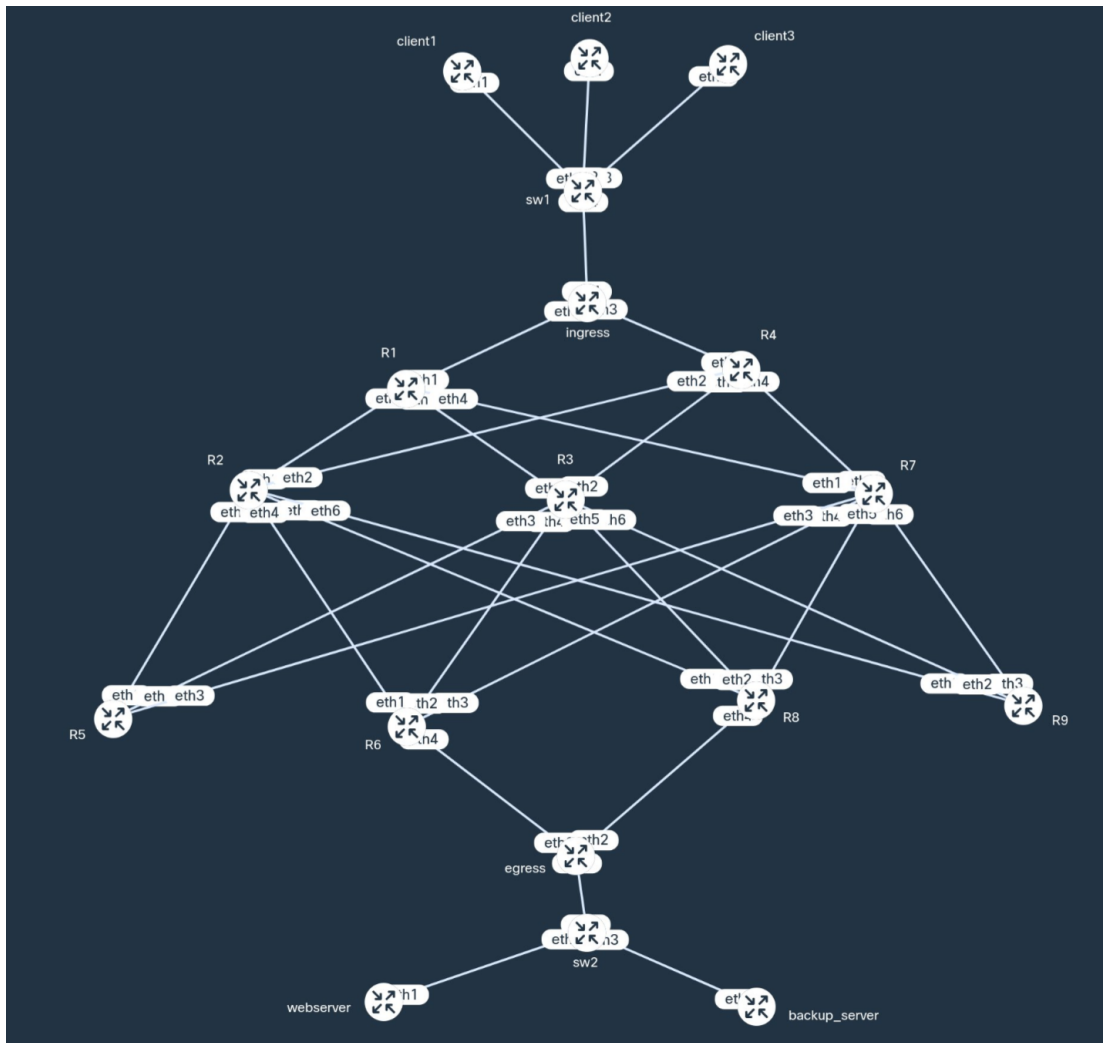


Figure 4.1 Clos architecture

4.3 Emulator overview

ContainerLab, Docker, and FRRouting (FRR) realise the network environment. These tools allow us to build a completely virtualised and manageable network. Docker is a lightweight, container-based virtualisation where each container is a network node independent of others. Linux containers don't emulate physical routers; they emulate routing devices. These containers are interconnected, and the overall topology (including the Segment Routing (SR) domain) is built using ContainerLab. Each container runs FRRouting as the routing software and has a command-line interface for configuration and management. This setup provides flexible deployment, easy modification of topology and repeatable experiments.

5 Tools and Pre-Tests

5.1 Tools

For the collection and analysis of network data, Wireshark and TShark are used. These tools allow analysis of protocol headers, forwarding behaviour, delay characteristics and, in particular, the overhead of the Segment Routing Header (SRH) in SRv6 traffic, which is a crucial part of this thesis. Traceroute is used to determine the path that packets follow through routers, which is useful to confirm routing decisions and SRv6 forwarding behaviour. Moreover, iproute2 services such as ping are used to check connectivity and measure the round-trip time (RTT) between nodes, which are important latency metrics for evaluating network performance. iPerf3 is a tool for active measurements of the maximum achievable bandwidth on IP networks. It is a traffic generator as well as a network performance measurement tool. The tool is designed to test the speed of a network connection by generating TCP or UDP data streams between two hosts. Instead of just considering the theoretical link performance, it is better to look at the real throughput, packet loss, jitter, retransmissions, latency effects, congestion behaviour and bandwidth utilisation. Engineers can run many different tests, such as single TCP flows, multiple concurrent streams, UDP stress tests, reverse direction traffic, long-term stability tests, small packet PPS tests, jumbo frame experiments and IPv4/IPv6 comparisons to detect bottlenecks caused by CPU limitations, queues, MTU variations, packet losses, TCP window size, RTT, QoS policies or buffer limitations.

5.2 pre-tests

To enhance visibility, please enlarge the PDF when figures appear.

The initial phase is the verification of SIDs advertised by ISIS protocols through CLI commands on the core routers within the SR domain. SIDs are effectively advertised by ISIS; refer to the figure below for verification.

For now, only three SIDs can be processed by the nodes. But with another mechanism, like binding SIDs, more SIDs can be used. SRH Max SL = 3 indicates that the router can process an SRH (Segment Routing Header) containing a maximum segment list depth of three SIDs. SRH Max End Pop = 3 specifies the supported limit for End/PSP-related SRv6 processing behaviours, while SRH Max H.Encaps = 2 indicates

```

ingress# show isis database detail | include SRv6
SRv6: 0:0
SRv6 Lan End.X SID: fc00:1000:1:1::, Algorithm: SPF, Weight: 0, Endpoint Behavior: End.X, Flags: B:0, S:0, P:0 Neighbor-ID: 0000.0000.0011
SRv6 SID Structure Locator Block length: 24, Locator Node length: 24, Function length: 16, Argument length: 0,
SRv6 Lan End.X SID: fc00:1000:1:2::, Algorithm: SPF, Weight: 0, Endpoint Behavior: End.X, Flags: B:0, S:0, P:0 Neighbor-ID: 0000.0000.0002
SRv6 SID Structure Locator Block length: 24, Locator Node length: 24, Function length: 16, Argument length: 0,
SRv6 Lan End.X SID: fc00:1000:1:3::, Algorithm: SPF, Weight: 0, Endpoint Behavior: End.X, Flags: B:0, S:0, P:0 Neighbor-ID: 0000.0000.0003
SRv6 SID Structure Locator Block length: 24, Locator Node length: 24, Function length: 16, Argument length: 0,
SRv6 Lan End.X SID: fc00:1000:1:4::, Algorithm: SPF, Weight: 0, Endpoint Behavior: End.X, Flags: B:0, S:0, P:0 Neighbor-ID: 0000.0000.0007
SRv6 SID Structure Locator Block length: 24, Locator Node length: 24, Function length: 16, Argument length: 0,
SRv6 Locator: fc00:1000:1::/48 (Metric: 0) standard
SRv6 End SID Endpoint Behavior: End, SID value: fc00:1000:1::
SRv6 SID Structure Locator Block length: 24, Locator Node length: 24, Function length: 16, Argument length: 0,

```

Figure 5.1 SRv6 over ISIS infos from Ingress CLI

the maximum supported SRv6 encapsulation nesting level. SRH Max End D = 5 represents support for multiple decapsulation-orientated End.DX/DT behaviours. These values are distributed through IS-IS TLVs so that all routers in the SRv6 domain are aware of the forwarding and processing limitations of peer nodes. This capability exchange is crucial for SRv6 traffic engineering, as excessively large SRHs or unsupported SID depths can result in packet drops, forwarding failures, or issues with policy installation.

```

ingress# show isis segment-routing srv6 node
Area CORE:
IS-IS L1 SRv6-Nodes:
IS-IS L2 SRv6-Nodes:

```

System ID	Algorithm	SRH Max SL	SRH Max End Pop	SRH Max H.encaps	SRH Max End D
0000.0000.0001	SPF	3	3	2	5
0000.0000.0002	SPF	3	3	2	5
0000.0000.0003	SPF	3	3	2	5
0000.0000.0005	SPF	3	3	2	5
0000.0000.0006	SPF	3	3	2	5
0000.0000.0007	SPF	3	3	2	5
0000.0000.0008	SPF	3	3	2	5
0000.0000.0009	SPF	3	3	2	5
0000.0000.0010	SPF	3	3	2	5
0000.0000.0011	SPF	3	3	2	5

Figure 5.2 SR capability of SR nodes

Another destination webserver configured with forced path policy with adjacency SIDs. Before SRv6 implementation, forwarding selects the path `client2` → `ingress` → `R1` → `R7` → `R8` → `egress` → `web-server` using SPF. After applying the SRv6 topology policy, an alternative path is selected based on the destination address, representing the first SRv6 implementation case. According to the bash file, the selected best path is `client1` → `ingress` → `R1` → `R3` → `R8` → `egress` → `web-server`. The results indicate that the raw throughput is very high, and although the delay is slightly higher, the overall stability remains good. Packet loss is lower, RED activation behaves more consistently, and TBF oscillation is minimal. Furthermore, TCP friendliness and queue consistency are significantly better when compared to the SPF-selected path.

Then packets are captured and recorded by Tshark and analysed by Wireshark for

better visuals. The first packets are captured from client1 while pingging the web server. As per the below figure, SRH packets couldn't be captured from outside of the SR domain.

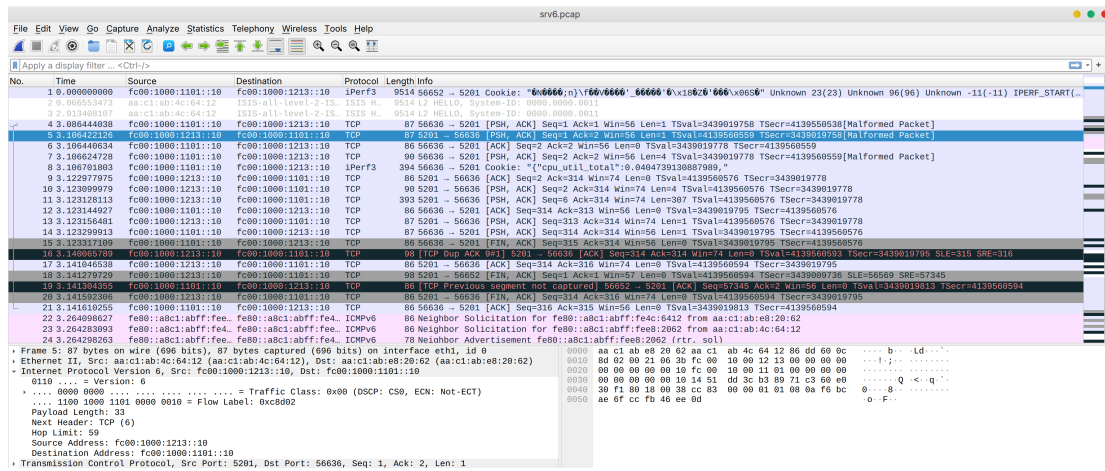


Figure 5.3 packets from outside

Next header is TCP (6). It confirms SRH is not part of the packet header. Next captured the packet from Ingress node which is part of trustable SR domain. SRH is part of the Packet header. Segment List is clearly visible and that contains detail information about path.

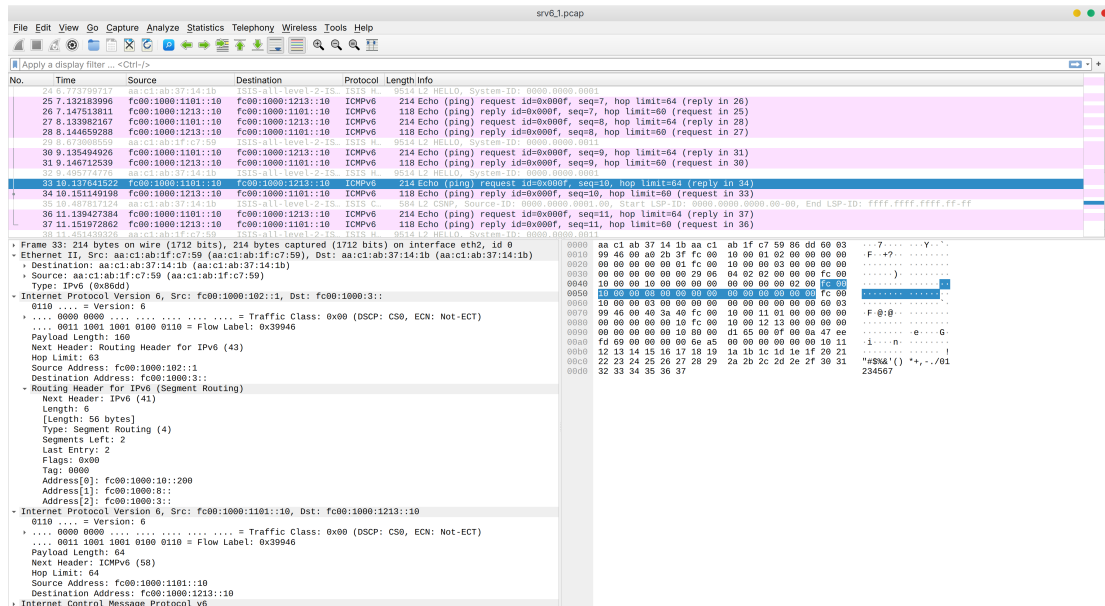


Figure 5.4 packets from Inside

The outer header provides the details of the Segment Routing Header (SRH), while the inner header encompasses the actual IPv6 packets, hence confirming effective encapsulation by the ingress. The Next header field (41) verifies the SRH. There are three SIDs represent two END and one END.DT6 behaviors.

Another SID has been added, although the maximum stop loss for SRH is just 3. Nodes can process more than three SIDs. ICMPv6 packets are transmitted successfully. The figure below elucidates SRH, facilitating a clearer comprehension of the RFCs. However, the RFC stipulates that nodes cannot process segments above the maximum segment list depth; however, this instance has exceeded that limit. Such instances may occur due to virtual routers or, occasionally, physical routers as well. This has to be clarified later.

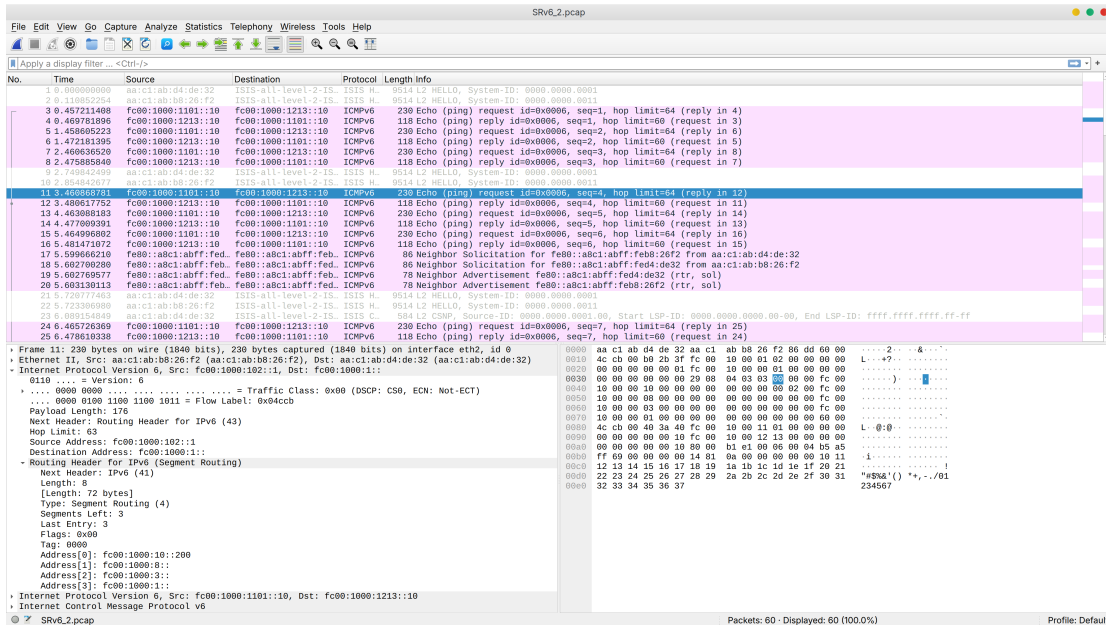


Figure 5.5 4 SIDs packet

5.3 Policy based programming

Policies must be established to enhance and systematise forwarding behaviour and network services. The initial policy prioritises a low-latency route over the default SPF-selected path. Therefore, traffic is forwarded through the path **Ingress** → R1 → R7 → R6 → Egress. This approach is referred to as the **low-latency policy** and is intended for delay-sensitive applications such as voice and video communication services.

The second policy is the **congestion mitigation policy**. This policy is designed to avoid bottlenecks caused by congested interfaces on specific nodes. During congestion conditions, traffic avoids nodes such as R2 and R6, and is instead redirected through R7 and R8. This demonstrates adaptive traffic engineering using SRv6.

The third policy focuses on **high-bandwidth forwarding**. The topology contains variable TBF rates across different links, allowing the evaluation of traffic engineering

decisions based on available bandwidth. This policy selects paths with maximum throughput, minimal RED activation, and lower queue accumulation in order to improve performance stability.

The fourth policy introduces **service chaining**. This stage highlights the programmability capabilities of SRv6. Additional network services such as a firewall and an Intrusion Detection System (IDS) can be integrated into the topology. Service nodes such as R9 or R5 can be explicitly inserted into the forwarding path so that packets traverse required network functions before reaching the destination.

The fifth policy is the **failover policy**. For example, if the primary path $R1 \rightarrow R7 \rightarrow R8$ fails, traffic is automatically redirected through the backup path $R1 \rightarrow R3 \rightarrow R6$. This policy enables a comparison between conventional SPF reconvergence and explicit SRv6 steering mechanisms.

Furthermore, multiple clients access the servers simultaneously using different traffic requirements. Applying a single policy to all clients may not provide optimal performance. For example, **Client 1** may use the low-latency policy, while another client may use the high-throughput policy. This approach is referred to as **multi-class traffic engineering**, where forwarding behaviour is adapted according to application or client requirements.

Finally, micro SIDs (**uSIDs**) can be implemented to evaluate SRH size reduction, packet overhead optimisation, segment scalability, implementation restrictions, and potential unexpected behaviours within SRv6 forwarding operations.

6 Ideas and Next Step

The next part of this thesis will focus on improving the current SRv6 implementation. The objective is to examine the capabilities of SRv6 in programmable networking and to identify the practical challenges that may arise during deployment. So far, we have successfully demonstrated SRv6 forwarding and SRH encapsulation, and we have controlled packet paths through SID-based forwarding. In addition, the basic uSID functions have been verified. However, further experiments are required to obtain a deeper understanding of SRv6's behaviour as specified in RFC 8986. The next stage involves analysing the behaviour of the SRv6 endpoints and their control mechanisms within the FRRouting framework. This includes the evaluation of functions such as End.X, End.DT6, and PSP, together with more advanced SR policy procedures. Packet capture and forwarding trace tools will be used to analyse SID usage, segment forwarding behaviour, and SRH processing. Traffic behaviour under policy-based forwarding at different stages will also be examined. Another important objective is to identify the practical limitations of FRRouting and Linux-based SRv6 implementations. The capabilities defined in the RFC specifications will be compared with the functionality currently supported by open-source routing technologies. Particular attention will be given to SR policy limitations, SID support, and maximum SRH depth. The dependency on the Linux kernel and the resulting implementation constraints will also be evaluated. The next phase will involve performance evaluation using tools such as iPerf3, Wireshark, and TShark. Metrics including throughput, delay, packet loss, and SRH overhead will be measured. These results will be compared between standard SRv6 forwarding and uSID-based forwarding approaches. In addition, SRv6 scalability improvements through uSID compression will be investigated. The impact of SID compression on SRH size, forward overhead, and forwarding performance in large SR domains will be analysed. Complex network scenarios will also be tested to evaluate deterministic forwarding and service chaining capabilities. Finally, potential architectural improvements for overcoming the identified limitations will be explored. This includes the use of eBPF-based packet processing, automated SR policy orchestration, and telemetry-informed traffic engineering strategies to improve the programmability and scalability of SRv6. The overall objective is to enhance the practicality and effectiveness of SRv6 in open-source networking environments.

Bibliography

- [1] Bob Hinden and Steve Deering. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 2460. 1998. URL: <https://datatracker.ietf.org/doc/html/rfc2460> (visited on 04/29/2026).
- [2] Clarence Filsfils et al. *Segment Routing Architecture*. RFC 8402. 2018. URL: <https://datatracker.ietf.org/doc/html/rfc8402> (visited on 04/29/2026).
- [3] Clarence Filsfils et al. *IPv6 Segment Routing Header (SRH)*. RFC 8754. 2020. URL: <https://datatracker.ietf.org/doc/html/rfc8754> (visited on 04/29/2026).
- [4] Clarence Filsfils et al. *Segment Routing over IPv6 (SRv6) Network Programming*. RFC 8986. 2021. URL: <https://datatracker.ietf.org/doc/html/rfc8986> (visited on 04/29/2026).
- [5] Maede Davoudzade et al. “Enhancing Network Simulation Accuracy through Calibration and Model Optimization”. In: *Proceedings of the Workshop on Performance Evaluation of Modeling and Simulation of Computer and Telecommunication Systems*. Ottawa, Canada, 2024.
- [6] Kevin Mills and Chris Dabrowski. *The Need for Realism when Simulating Network Congestion*. NIST Publication. 2016. URL: <https://www.nist.gov/publications/need-realism-when-simulating-network-congestion> (visited on 05/09/2026).
- [7] D. W. Ngwenya, M. C. Hlophe, and B. T. Maharaj. “TCP Congestion Control Algorithm Using Queueing Theory-Based Optimality Equation”. In: *Electronics* 14.2 (2025), p. 263. DOI: [10.3390/electronics14020263](https://doi.org/10.3390/electronics14020263). URL: <https://www.mdpi.com/2079-9292/14/2/263> (visited on 05/09/2026).